
ARTICLE

Optimized Sequence Recognition Framework for Memory-Assisted Threat Mitigation in Wireless Intrusion Environments

K. Padmavathi^{1,*} and Basheer Riskhan²

¹Associate Professor, Department of Computer Science, PSG College of Arts & Science, Coimbatore, India.

²Dean and Associate Professor, School of Computing and Informatics, Albukhary International University, Malaysia.

*Corresponding Author: K. Padmavathi. Email: padmasakthivel@gmail.com

Received: 09 June 2024; Accepted: 20 September 2024; Published: 30 December 2024

ABSTRACT: In order to achieve high security in the networking domain are considered in current scenarios, various methods have been proposed based on software for pattern matching. But Approaches that are based on software do not provide adequate scalability and reconfigurability to today's level of security provisioning. To give enough security in a Wireless Intrusion Detection System, this project work moves towards the hardware-based approaches. It provides a high level of security. Hardware-based approaches mainly concentrate on memory efficiency in pattern matching. In this project work, a new algorithm was proposed based on hardware approaches, which provides an efficient memory for pattern matching. The state traversal pattern matching algorithm provides a significant reduction in the memory used when compared with that of the conventional Aho-Corasick algorithm. This new algorithm produces memory reduction through a new design methodology, which constructs the state traversal machine with a size of 1280bytes. Within 1280 bytes, the users can store large-sized string patterns in the pattern database and easily retrieve them from the database using the state traversal machine through the path vector. The previous algorithms need a huge amount of memory compared to the conventional Aho-Corasick algorithm and bit split algorithm. In addition, the number of basic ASCII characters is 128. We can design the memory architecture using a Binary Search Tree (BST) structure. Each node required 10 bytes of memory space. So, it is needed a very few amounts of memory space is needed, such as 128(ASCII character) X10 is 1280 bytes. Our hardware generates the address of input strings, and all the common addresses of input strings are merged by the state traversal machine and the bits split algorithms. The merged addresses of input strings are encrypted and decrypted by the Blowfish algorithm to allow the valid packets or discard the invalid packets using Wireless and Detection Systems (WIDS).

KEYWORDS: WIDS, Pattern Matching, Binary Search Tree, AC Algorithm, and Finite Automata.

1 Introduction

Intrusion detection is an approach to security that is also evolving towards networked environments. Initially, Intrusion detection is a technique for unearthing masqueraders and misfeasors in standalone systems; later, it is slowly shifted towards networks. In a networked systems environment Intrusion Detection System (IDS) is the common source that protects the systems from attack or malicious perversion [1]. The IDS development and distribution are exhibited in terms of equipment, protocols, connectivity, cost, and automation. This is highly evident in Wireless Intrusion Detection Systems (WIDS), which hold many chances and challenges connected



with data transmission through an open, mutually shared medium. The operation of a WIDS is a multistep process from the origination of an attack through to human human-readable assessment. If an optimum solution is needed, then a precise consideration has to be provided to each process in the chain from attack detection. In recent days, research has focused on each discrete facet of a WIDS with trivial consideration of the entire system operation. In recent days, research has focused on each discrete facet of a WIDS with trivial consideration of the entire system operation. Taking a holistic view of the technology shows the interconnectivity and inter-dependence between stages, leading to improvements and novel research areas for investigation [2]. This chapter will outline the general structure of Wireless Intrusion Detection Systems and briefly describe the functions of each improvement stage, categorized into the following six areas: Architecture, Threat Identification, Intrusion Detection, Data Collection, Alert Correlation, Evaluation. These substances are considered crucial topics for beginners. Attention will be placed on developing WIDS' future stages by supporting the readers to be aware of the impact made at the initial stages. WIDS is a wider concept than catching break-in attempts. It also includes verifying the access points that are on the network, identifying any that should not be there or have security issues, and detecting attacks on access points or clients [3].

Encryption is the method by which the plain text is converted into something that seems to be a meaningless and random subject. Decryption is the method that changes the ciphertext generated through encryption back to the original text. Symmetric encryption is the process of encrypting a large quantity of data. In order to decrypt the data, the same key that was used for its encryption is to be used. The primary objective of encryption is to hide the original text and meaning behind something that seems meaningless by making the key a mandate to decrypt the same. The challenge is to develop an encrypted text that has no or a very minimal chance of being decrypted without the key [4]. An encryption is termed as good when there is no other option other than trying all the possible keys in order to decrypt the same. For such encryptions, the longer the key, the larger the number of possible keys that need to be tried to decrypt. The quality of encryption is a very important task to be considered, as even the strongest algorithm can be broken with a strong attacking method. One needs to be very careful in selecting an encryption standard, and mostly an algorithm that has a longer life and high resistance towards attacks to be chosen. In this memory architecture, the string patterns are taken as input, which is compiled by a finite state machine; as a result, substrings that match to string patterns are detected. Then that particular string pattern transition table is stored in memory [5].

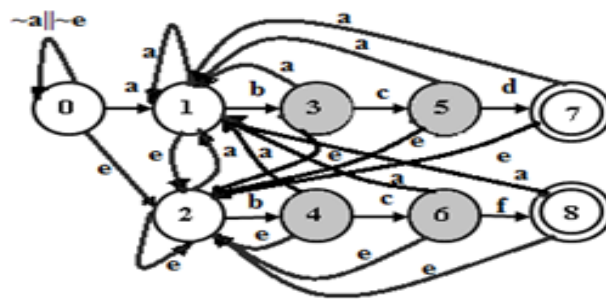


Figure 1: DFA for matching “abcd” and “ebcf”

Fig. 1 depicts the state transition diagram that accepts the input strings “abcd” & “ebcf”. States 7 and 8 are final states or accepting states for the input string “abcd” & “ebcf”. Two strings abcd, ebcf have having common string ‘bc’. For character ‘b’, two similar transition states 3 & 4 are there in FSM, and for character ‘c’ also two similar transition states 5 & 6 are there in FSM. So, it leads to an increasing number of states, in other words, memory. In order to achieve the reduction of memory size, the two states are combined, i.e., 34 is the accepting state of character ‘b’ & 56 is the accepting

state of character 'c'. In this study, combining similar states is done using by linked list data structure. The algorithm used to reduce FSM states and memory size is the Merged List. Fig.2 shows the memory architecture used to instrument the finite state machine. In the presented architecture, the current state and an input character are collected in the Memory Address Register (MAR). This memory address represents the memory location of that particular input character. In this, with the use of match vector information, the string matching is found. The match vector value is zero represents that no substrings are matched. Otherwise, the match vector has a value. For example, an input character is f, and the current state is 6. In this case next state is 8, and the match vector is 2 because "ecdf" is matched [6].

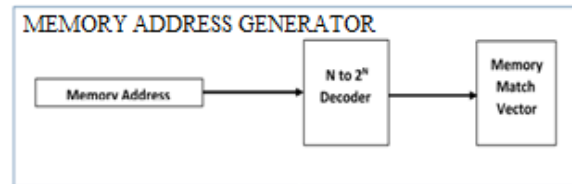


Figure 2: Basic memory architecture

In order to achieve high network security, Networks should use an IDS. Through Memory architecture, scalability and reconfigurability are achieved in IDS. In this study, a novel algorithm based on hardware approaches, which provides an efficient memory for pattern matching by decreasing the FSM states than the Original FSM states. Comparing the classic Aho-Corasick algorithm, the state traversal pattern matching algorithm provides 21% of memory reduction. In addition, this provides 24% of memory reduction when combined with the bit-split algorithm. Our new algorithm produced to requires a very small amount of memory compared to the traditional Aho-Corasick algorithm. It does not provide security in previous algorithms, including the Aco-Corasick algorithm and the bits split algorithm. We propose both aspects, such as memory reduction as well as data security.

2 Related Works

This section briefs the research works associated with this project. In recent research number of algorithms based on hardware approaches have been proposed. Generally, hardware approaches are based on logic and memory architectures. The hardware-oriented approaches are subdivided into a couple of architectures based on the memory and logic. The architecture that is based on logic is operated in logical resources by on-chip in a field-programmable gate array (FPGA) that is subjected to making changes in the pattern of regular expression on a state machine that are parallel or through circuits, since the FPGA has the drawback of allowing fresh patterns of attacks. Idhu et al. [7] have developed an algorithm in order to compile the patterns of regular expressions into a form of circuits that are combined based on a nondeterministic finite automaton (NFA). Hutchings et al. [8] proposed a module generator that can employ the prefixes that are common to reduce the area of the circuit in an FPGA. Moscola et al. [9] introduced a module that is based on the scanning of the content that is applied on a firewall of an internet service with the aid of an FPGA. Clark et al. [10] put to developed an area and the throughput with the inclusion of pre-coded inputs that are wide and parallel to the conventional NFA applications. Baker et al. [11] introduced a pre-decoded multiple-pipeline shift-and-compare matcher by changing incoming characters into many bit lines, which minimized the routing complexity and comparator size. Lin et al. [12] designed a sharing architecture with the motive of reducing circuit areas by employing common infix and suffix sub-patterns.

According to re-configurability and scalability, memory architectures are highly focused because memory is flexible and scalable. In the context of matching multiple patterns, the Aho-Corasick (AC) algorithm [13] is highly suitable and a popular algorithm. Aldwairi et al. [14] suggested a configurable

string-matching accelerator by employing a memory implementation of the AC FSM. Tan et al. [15] designed an algorithm called the bit-split algorithm for partitioning a large AC state machine into small state machines with the motive of minimizing the memory needs. Jung et al. [16] proposed an architecture by utilizing an FPGA implementation of the bit-split string matching. Piyachon et al. [17] designed a machine intending to reduce the memory size by relabeling states of the AC state machine. Piyachon et al. [18] planned a method by employing Label Transition Table and CAM-based Lookup Table to significantly reduce the memory size. Cho et al. [19], [20] methodized a processor that is used to store the list of substrings and the state transitions based on hash-based pattern matching. Dharmapurikar et al. [21] designed an algorithm for pattern matching, which is used to consider multiple characters at a time by utilizing the modified AC algorithm. Likewise, the content-addressable memories (CAM) are also broadly used for string matching, since they match the entire pattern at a time when the pattern is shifted past the CAM. Gokhale et al. [22] utilized the CAM to execute a parallel search at a high speed. Sourdis et al. [23] employed the pre-decoded technique for the CAM-based pattern matching to minimize the area. Also, Yu et al. [24] designed a pattern-matching called ternary content-addressable memory (TCAM)-based multiple-pattern matching, which could manage complex patterns, correlated patterns, and patterns with negation. The hash-based approach is methodized to employ a Bloom filter for deep packet inspection. Dharmapurikar et al. [25] planned a mechanism by employing the parallel bloom filters called a hashing-table lookup mechanism to allow a large number of fixed-length strings for hardware scanning. Lockwood et al. [26] suggested a smart gateway grounded on a Bloom filter that facilitates Internet worm and virus protection in both wide area networks (WAN) and local area networks (LAN).

3 Proposed Methodology

The most popular and widely used algorithm for multiple string matching is the Aho–Corasick (AC) algorithm. The AC algorithm minimizes the number of states in the FSM. This algorithm utilizes a Finite State Machine. Dotted lines are represented for failure transition, and solid lines are represented for valid transition in the AC algorithm. In the AC algorithm, valid transitions are represented by solid lines, and Failure transitions are represented by dotted lines. This algorithm is achieved by the Match vector. The AC machine checks the valid transition for the input character. Suppose not valid, then the state machine represents a failure transition. Otherwise, take another input character to check valid character or not. In Fig. 3, the AC Machine is in State 0, in this machine accepts “study” & “crude”. For the input character ‘s’ AC Machine moved to state 1. For the input character ‘t’ AC Machine moved to state 2. For the input character ‘u’ AC Machine moved to state 3. For the input character ‘d’ AC Machine moved to state 4. For the input character ‘y’ AC Machine moved to state 5. For the input character ‘c’ AC Machine moved to state 6. For the input character ‘r’ AC Machine moved to state 7. For the input character ‘u’ AC Machine moved to state 8. For the input character ‘d’ AC Machine moved to state 9. For the input character ‘e’ AC Machine moved to state 10. In this process, the same input character is processed many times from state 0.

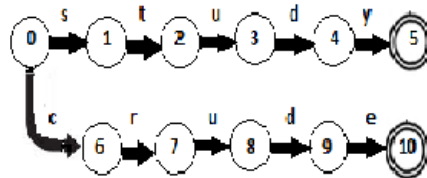


Figure 3: State diagram of an AC machine

In Fig.3, the double circle nodes indicate the accepting state. For the string “study”, the AC state table stores the match vector {10} & accepting state 5. For the string “crude”, the AC state table stores the match vector {01}. Match vector {10} & {01} represents the accepting state/ final state. Match vector {00} is not an accepting state. So, the AC algorithm is compiled & stores the same substrings many times. So, to achieve the speed and reduce memory to store substrings, in the AC algorithm same substrings that are repeatedly compiled and stored should be eliminated.

	Input	Next state	Failure	Match vector
State 0:	S	1	0	00
State 0:	C	6	0	00
State 1:	T	2	0	00
State 2:	U	3	0	00
State 3:	D	4	0	00
State 4:	Y	5	0	10
State 6:	R	7	0	00
State 7:	U	8	0	00
State 8:	D	9	0	00
State 9:	E	10	0	01

Figure 4: AC state table

A state traversal pattern matching algorithm substantially lessens the memory needs by creating a memory-efficient pattern-matching algorithm. This algorithm captures 21% of memory reduction when compared with the classical Aho–Corasick algorithm. This state traversal pattern matching algorithm is used to increase the throughput of the network & increase the security of the data in the Network Intrusion Detection System (NIDS). So that new algorithm should beat this state traversal pattern matching algorithm by reducing the memory for exact string patterns.

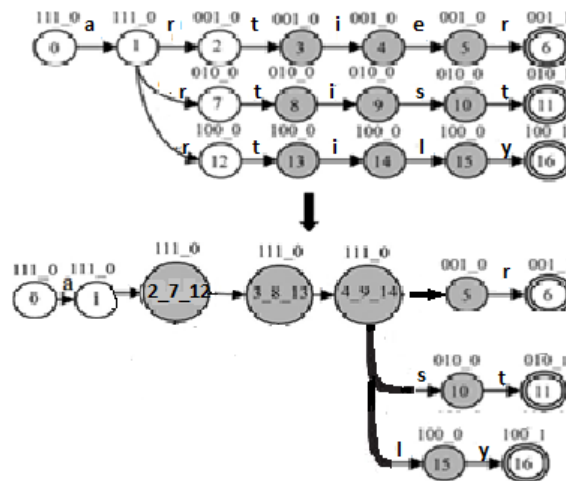


Figure 5: A state traversal pattern matching algorithm

The state traversal machine comprises: 1) the construction of valid transition, failure transition, pathVec, and iffFinal functions, and 2) merging pseudo-equivalent states. The states 0 to 6 match the first pattern “artier”. Hence, all states of pathVec are set to 001, and to notify the final state (path terminates), the iffFinal of state 6 is set to 1. Since there is an edge labeled *a* from state 0 to state 1, when a pattern called “artist” is added to the graph, the edge is reused. The pathVec of other states, such as state 7, state 8, state 9, state 10, state 11, on the path is set to 010, and the pathVec of states 0 and 1 is set to 011. To indicate the final state for the second pattern, the iffFinal of state 11 is set to 1. Likewise, when the third pattern (“artily”) is appended to the graph, the pathVec of states (0,1) is set to 111. The other pathVec states, which are state 12, state 13, state 14, state 15, and state 16 on the path, are set to 100. The third pattern final state (iffFinal of state 16) is set to 1. Further, the Ac

algorithm abstracts and combines the pseudo-equivalent states. Merging pseudo equivalent states comprises merging the failure transitions and performing the union on the pathVec of the merged states. This algorithm has some demerits. In order to store the 3 strings, this algorithm wants a 3-bit path vector for each string. Suppose for storing n strings, this algorithm wants n bits of length path vector for each string. So the path vector combinations want a separate memory space. In order to reduce those problems, this new algorithm builds a new memory architecture. Through that, memory efficiency is achieved.

3.1 Our New Hardware Memory Generator

In order to achieve the speed & reduce the memory to store the string, in the AC algorithm same substrings that are repeatedly compiled and stored should be eliminated. To avoid that, much research has been completed. From that, memory efficiency is achieved, but not enough for today's networking. In this study, discuss about to give more memory efficiency through a new algorithm. In this new algorithm, a new state traversal machine is implemented. Normally string has only letters. Each character wants only one byte to store in memory. This new design methodology only stores the 26 alphabets. Each character is connected with all the other characters using of linked list.

In this work, a new algorithm was proposed based on hardware approaches, which provides an efficient memory for pattern matching. Already State traversal pattern matching algorithm supplies memory reduction when compared with the classic Aho-Corasick algorithm. This new algorithm produces memory reduction through a new design methodology, which constructs the State traversal machine with a size of 128 characters are requiring 128X10 bytes. Each character node requires 10 bytes of memory space. Totally, we designed the memory hardware size to be 1280 bytes. The memory hardware is generating the address of the user's input strings.

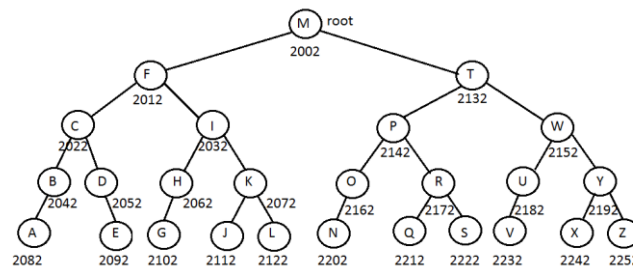


Figure 6: Alphabetically Order with Memory Address Generator

The main thing in this project work is the construction of a state traversal machine. This tree contains characters from character 'A' to 'Z', a total of 26 characters is stored. The character 'M' is assigned as the root node. The smaller than character 'M' is arranged in the left sub-tree. The greater than character 'M' is arranged in the right sub-tree. The memory hardware generates the address of input strings. Suppose the input string is "EGJL" and the memory hardware generates the addresses of the equivalent input string are 2092, 2102, 2112, and 2122. Here memory address of character 'E' is 2092, character 'G' is 2102, character 'J' is 2112, and character 'L' is 2122. Each node requires 10 bytes of memory space. In this, addresses are collected and construct the finite state traversal machine (FSM).

3.2 Formation of New State Traversal Machine

This module is used to construct a state traversal machine that contains the address of input strings. Suppose the two input strings such as "WXYZ" and "PXYQ". The address of the input string "WXYZ" is address 2152 for the character 'W', address 2242 for the character 'X', address 2192 for the character 'Y', and address 2252 for the character 'Z'. Similarly, the address of the input string

“PXYQ” is address 2142 for the character ‘P’, address 2242 for the character ‘X’, address 2192 for the character ‘Y’, and address 2212 for the character ‘Q’. The traversal machine is creating two paths. First path binary number starts with “01_0” where the first two bits are Path_Vector and the last bit indicates the start or middle, or last state. Similar to the second path binary number starts with “10_0”. Final state is set the bit as in the first path is 01_1. Others are set as bit is 01_0.

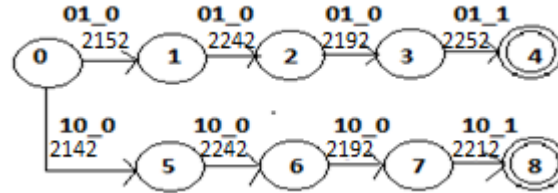


Figure 7: State Traversal Machine

Both strings “WXYZ” and “PXYZ” have the common characters ‘X’ and ‘Y’. The input character ‘X’ address (2242) transitions from state 1 to 2. Similarly, the input character ‘X’ address transitions from state 5 to 6. States 2 and 6 are merged by using the merged Finite State Machine algorithm. Similarly, the input character ‘Y’ is present in both strings. So, the states 3 and 7 are merged. After merging the input characters ‘X’ and ‘Y’.

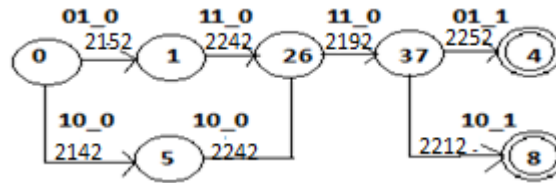


Figure 8: Merged State Traversal Machine

This is an important process in this research. Because with the use of this module, only the memory will be reduced to extract the string patterns. In this module, similar states of different strings are merged to achieve memory efficiency. Normally, all characters of a string are stored in separate locations, which leads to an increase the memory as well as an increase in finite state machine size. This architecture reduces the memory for a string and reduces the FSM size.

Pseudo-equivalent states –definition: If two states are said to be pseudo-equivalent states, they have different next states, but alike transitions, same failure transitions, and same iffFinal bit.

In Fig. 7, since the 2nd and 6th states consist of the same input transitions, same failure transitions to 0th, and same iffFinal bit 0, they are said to be pseudo-equivalent states. Also, the 3rd and 7th states are in the same situation. In the algorithm, the pseudo-equivalent states 2 and 6 are combined to be state 26, and states 3 and 7 are combined to be state 37, as shown in Fig. 8. The pathVec_iffFinal of the merged states are used to updated by taking the union of the pathVec_iffFinal. For this reason, the pathVec_iffFinal of states 26 and 37 are changed to be {11_0}. Further, a register is needed called preReg, to track the precedent pathVec in every state. The width of preReg and pathVec is equal. Every bit of the preReg corresponds to a string pattern. By performing a bitwise AND operation, the preReg is updated in each state on the pathVec of the next state and its current value. All the merged states can be differentiated by tracking the preceding path entering into the merged state. The value of the preReg displays the match vector of the matched pattern while reaching the final vector. The machine reaches the failure mode and selects the failure transition as in the AC algorithm, during the state traversal, and if all the bits of the preReg become 0. Thereafter, any failure transition, all the bits of the preReg are reset to 1.

state	0	5	26	37	0
input char		w	x	y	z
pathVec	11	10	11	11	01
preReg	11	10	10	10	00
ifFinal	0	0	0	0	1

Figure 9: Transition function of string “wxyz”

Consider an example in Fig. 9 where the string “pcdf” is applied. Firstly, the preReg is initialized to $\{P_2P_1\}=\{11\}$. After taking the input character ‘w’ in state 0, the merge_FSM reaches state 5 and updates the preReg by executing a bitwise AND operation on the pathVec {10} of state 5 and the current preReg {11}. The resulting new value of the preReg will be $\{P_2P_1\}=\{10\text{AND}11\}=\{10\}$. Next, by taking the input character ‘x’, the merge_FSM reaches the state 26 and updates the preReg by doing a bitwise AND operation on the pathVec {11} of state 26 and the current preReg {10}. The preReg maintains $\{P_2P_1\}=\{11\text{AND}10\}=\{10\}$. Next, by taking the input character ‘y’, the merge_FSM reaches state 37 and updates the preReg by doing a bitwise AND operation on the pathVec {11} of state 37 and the current preReg {10}. The preReg remains $\{P_2P_1\}=\{01\text{AND}10\}=\{00\}$. Lastly, the merge_FSM goes to state 4 after getting the input character ‘z’. Then the preReg becomes $\{P_2P_1\}=\{01\text{AND}10\}=\{00\}$, after performing a bitwise AND operation on the pathVec {01} of state 4 and the current preReg {10}. As per the algorithm, the machine will go to the failure mode if all the bits of the preReg become 0 during the state traversal.

state	0	5	26	37	8
input char		p	x	y	q
pathVec	11	10	11	11	10
preReg	11	10	10	10	10
ifFinal	0	0	0	0	1

Figure 10: Transition function of string “pxyq”

Likewise, consider another example in Fig. 10 where the string “pxyq” is employed. The process of state traversal is the same as the previous example, as for as the state machine reaches state 37 and the input character is ‘q’. Then, by taking the input character ‘q’, the merge_FSM reaches state 8, and the preReg becomes $\{P_2P_1\}=\{10\}$ by doing a bitwise AND operation on the pathVec {10} of state 8 and the current preReg {10}. Since the value of ifFinal is 1, the value of preReg, $\{P_2P_1\}=\{10\}$, indicates that the pattern P_2 is matched.

Algorithm: state traversal pattern matching algorithm
 Input: input string as x=“abcd” and state traversal machine
 M to find valid transition function or invalid transition function using pathvec tor algorithm.
 Output: Identify at which key words occur in x.
 Method:
 Step1: state=0
 Step2: prereg=1...1 //assign all bits are 1...1 to prereg
 Step3: for(i=0;i<n;i++)//n as length of the string
 Step4: begin
 Step5:prereg=prereg&&pathvector
 Step6: if(prereg!=0) then goto step7 otherwise goto step8
 Step7: Read input character
 Step8: set prereg=1...1
 Step9: ifFinal(state)==1 then
 Step10: print valid transition and exit
 Step11: end
 Step12: print invalid transition.

Figure 11: Algorithm for State Traversal Pattern Matching

In Figure 12, similar states of strings “abcdef”, “apcdeg”, and “awcdeh” are merged. The ‘cde’ substring is common for all three strings. So, these three states of different strings are merged. In the

above figure, for the common substring 'c', the states 3,8,13 are merged. For the common substring 'd', the states 4,9,14 are merged. For the common substring 'e', the states 5,10,15 are merged. Normally, each character occupies 1byte of space in memory. In this example, for storing these 3 strings, the 'd' character wants 3 bytes. After merging all the strings, only 1byte of memory space is enough for the character 'd'. All three strings access the same memory space to access the character 'd' with the help of FSM or memory architecture. So, it will be useful for reducing memory for storing and extracting the string. As a result, all the strings are using common memory for substrings in a new state traversal machine. This is also one of the methods to reduce the memory.

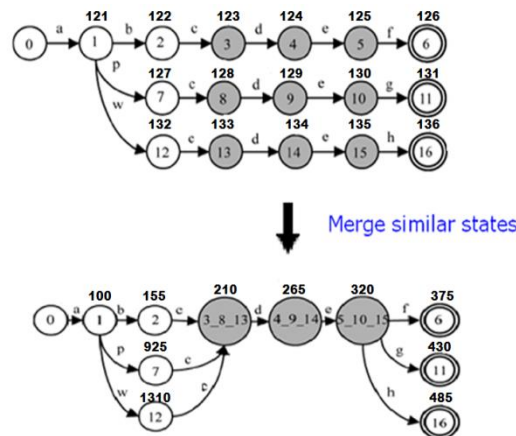


Figure 12: Merging the similar states

3.3 Loop Back in Merged States

In a state machine, some cases of multiple sections of pseudo-equivalent states are combined, and this may create a loop-back problem. The loopback problem is due to combining common sub-patterns with dissimilar sequences. For instance, consider the two patterns, "pqrstu" and "wstqrv" have the common sub-pattern, "qr" and "st", exhibited in different sequences. The corresponding state machine is shown in Figure a. Since the common sub-patterns, "qr", in the 2nd state, 2, and 10th state, 3rd and 11th States are illustratively equivalent. The sub-patterns, "st", the 4th and 8th states, 5th and 9th states are also illustrative equivalents.

Figure b depicts the loop-back transition from the 5th to the 2nd state. The loop transition may cause results that match as false positives. For instance, the input string "aqrstqrstu" will be flawed as the pattern "pqrstu". Loop back transitions will not occur as long as the common substrings appear in sequence while merging the corresponding pseudo-equivalent states. That is, the pseudo-equivalent states can be combined without loop-back issues if the common sub-patterns appear in sequence. As a result, in the program, the order of common sub-patterns is recorded and identified.

Algorithm: Extract substring without loop back
 Input: Two string patterns
 Output: Shared substrings
 Method:
 Step1: Using LCS algorithm to extract common substrings of the two string patterns.
 Step2: Label such as α, β, γ and δ the common substrings as new sequences.
 Step3: The new sequences are avoid the loop back problems.
 Step4: Output substrings are same sequence.

Figure 13: Algorithm for extracting substrings

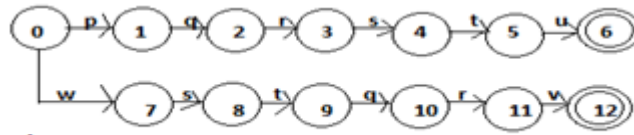


Fig. a

Figure 13a: Corresponding state machine

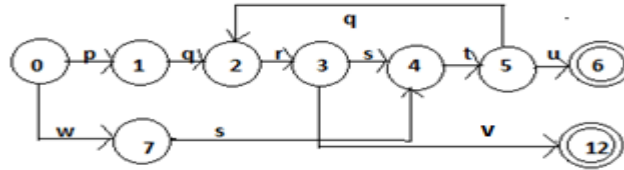


Figure 13b: The loop-back transition

Fig. b depicts the pseudo code of the algorithm to detect the common substring without loopback issues. Primarily, by employing the longest substring algorithm, common substrings are extracted. The execution results will report all the possible common substrings. Then, all the sequences of common substrings are labeled as a new sequence. Next, the longest common subsequence (LCS) algorithm is utilized to detect all the common string's longest subsequence. The outcome of LCS ensures zero loop backing in transitions. For instance, let the two substrings pattern can be extracted, "nopqrstuvwxyz" and "nopabqrstuvwxyz".

By employing the longest common substring algorithm, all the common substrings can be extracted, which are "nop", "qrs", "tuv", and "xyz". Then, we label the substrings "nop", "qrs", "tuv", and "xyz" as α , β , γ , and δ , respectively. Therefore, the sequence of substrings in "nopqrstuvwxyz" is labeled as " $\alpha\beta\gamma\delta$ " while the sequence of substrings in "nopabqrstuvwxyz" is labeled as " $\alpha\beta\gamma\delta$ ". Next, the LCS algorithm is utilized in order to find all the longest common subsequences in between the two new sequences, " $\alpha\beta\gamma\delta$ " and " $\alpha\beta\gamma\delta$ ", and the outcomes are " $\alpha\beta\gamma\delta$ " or " $\alpha\gamma\delta$ ". Hence, the subsequence α ("nop"), γ ("tuv"), and δ ("xyz") or the subsequences of α ("nop"), β ("qrs"), and δ ("xyz") can be combined without loop-back issues. Notice that the outcome of LCS has a non-unique nature. In Section IX, we will define a gain function to select subsequences to be combined.

3.4 Encrypt and Decrypt Merged data using Blowfish algorithm

By employing the blowfish algorithm, the input strings are merged. Since the Blowfish algorithm is a symmetric encryption algorithm, it uses the same secret key to both encrypt and decrypt messages. It is also known as a block cipher algorithm, as it splits a message into fixed-length blocks during encryption and decryption. The block length for Blowfish is 64 bits. Encryption is a security method that allows only an authorized person to read the message. In order to achieve encryption, an encryption algorithm is employed to produce ciphertext. The encrypted messages can be read only if they are decrypted. Figure. 14 is an encryption algorithm that generates ciphertext. Figure. 15 is a decryption algorithm that generates actual text.

```

*plaintext_string = "WXYZ PXYQ";
printf("Encrypted message string is: ");
while (plaintext_len)
{
    msg_left = msg_right = 0UL;

    for (block = 0; block < 4; block++)
    {
        msg_left = msg_left << 8;
        if (plaintext_len)
        {
            msg_left += *plaintext_string++;
            plaintext_len--;
        }
        else msg_left += 0;
    }
    for (block = 0; block < 4; block++)
    {
        msg_right = msg_right << 8;
        if (plaintext_len)
        {
            msg_right += *plaintext_string++;
            plaintext_len--;
        }
        else msg_right += 0;
    }
    Blowfish_Encrypt(&ctx, &msg_left, &msg_right);
    printf("%lx%lx", msg_left, msg_right);
    *ciphertext_string++ = (uint8_t)(msg_left >> 24);
    *ciphertext_string++ = (uint8_t)(msg_left >> 16);
    *ciphertext_string++ = (uint8_t)(msg_left >> 8);
    *ciphertext_string++ = (uint8_t)msg_left;
    *ciphertext_string++ = (uint8_t)(msg_right >> 24);
    *ciphertext_string++ = (uint8_t)(msg_right >> 16);
    *ciphertext_string++ = (uint8_t)(msg_right >> 8);
    *ciphertext_string++ = (uint8_t)msg_right;
    ciphertext_len += 8;
}

```

Figure 14: Encryption using the Blowfish Algorithm

```

printf("Decrypted message string is: ");
ciphertext_string = &ciphertext_buffer[0];
while(ciphertext_len)
{
    msg_left = msg_right = 0UL;
    for (block = 0; block < 4; block++)
    {
        msg_left = msg_left << 8;
        msg_left += *ciphertext_string++;
        if (ciphertext_len)
            ciphertext_len--;
    }
    for (block = 0; block < 4; block++)
    {
        msg_right = msg_right << 8;
        msg_right += *ciphertext_string++;
        if (ciphertext_len)
            ciphertext_len--;
    }
    Blowfish_Decrypt(&ctx, &msg_left, &msg_right);
    printf("%c%c%c%c%c%c%c%c",
        (int)(msg_left >> 24), (int)(msg_left >> 16),
        (int)(msg_left >> 8), (int)(msg_left),
        (int)(msg_right >> 24), (int)(msg_right >> 16),
        (int)(msg_right >> 8), (int)(msg_right));
}

```

Figure 15: Decryption using the Blowfish Algorithm

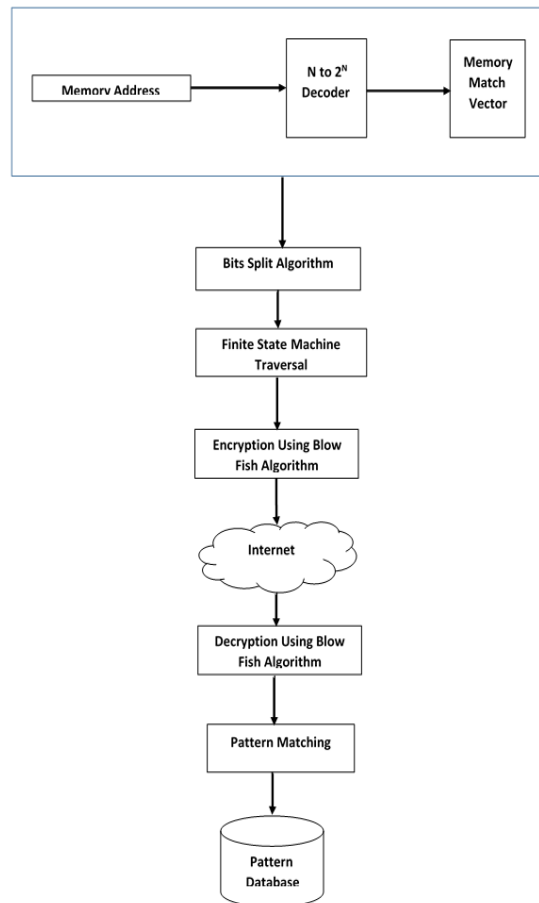


Figure 16: System Architecture

4 Experimental Results

The 2217 patterns of strings that hold 36539 number of characters are obtained from the database of all the rules in the version 2.4 of Snort set. The experiments are carried out with the AC algorithm first and then the bit-split. The flow in which the process takes place is depicted in Figure 17. During the early stages, the patterns are obtained from the aforesaid database. In the stage that follows, the patterns that belong to the 32 strings are segmented into a module that is based on the pattern similarity. In the subsequent stage, the LCS is used to obtain all the substrings, eliminating the issues that may occur due to the loopback. As there is non-uniqueness in the results obtained, the substrings that are common and that have the biggest sharing gain are considered and selected. The former is defined as the length of substrings that are in common multiplied by the number of patterns that share the substrings that are in common. For example, consider three patterns, “1common1”, “2common2”, and “3common3”, and they are identified to have the word “common” as a substring that is common to all three. The sharing gain will be $6 \times 3 = 18$ since the substring “common” has 6 characters that are shared among the three patterns. In the final stage, the common substrings are obtained are subjected to merging, and the transition table is generated.

The experimental results of the integration of the algorithm into the AC algorithm are shown in Figure 18. The name of the rule set is exhibited in Column 1, Column 2, and Column 3 of the Table. The number of state transitions, the number of states, and the memory size of the AC algorithm are expressed in Columns four, five, and six. The results of the approach are depicted in columns seven, eight, and nine. The tenth column indicates that there is a memory reduction than that of the AC

algorithm. Column eleven is our new algorithm with a merged finite state machine, as shown in the Figure 19. Table. Rules set Oracle the count of patterns 337, the count of characters 11128, the count of transitions 6793, the count of states is 6804, and the required memory in bytes is 49267. But the AC algorithm requires the memory size in bytes of 30699. But our algorithm counts the number of characters available in the patterns. Only 49 letters and special symbols are available. So it allocates 490 bytes of memory space. Similarly, Deleted allocates 780 bytes, Exploit allocates 560 bytes, Web-misc allocates 740 bytes, Smtip allocates 1120 bytes, Misc allocates 920 bytes, Ftp allocates 900 bytes, and other rule sets allocate 1180 bytes. The average memory requirement is 4% of AC and other algorithms. So, total memory reduction is 96% of other algorithms.

```

2.Display
3.Exit
1
Enter the new node data
m
Do you want to continue?Y/N
Enter the new node data
f
Do you want to continue?Y/N
Enter the new node data
c
Do you want to continue?Y/N
Enter the new node data
i
Do you want to continue?Y/N
Enter the new node data
b
Do you want to continue?Y/N
Enter the new node data
d
Do you want to continue?Y/N
Enter the new node data
h
Do you want to continue?Y/N
=
Enter the new node data
k
Do you want to continue?Y/N
Enter the new node data
a
Do you want to continue?Y/N
Enter the new node data
e
Do you want to continue?Y/N
Enter the new node data
j
Do you want to continue?Y/N
Enter the new node data
l
Do you want to continue?Y/N
Enter the new node data
t
Do you want to continue?Y/N
Enter the new node data
p
Do you want to continue?Y/N
Enter the new node data
w
Do you want to continue?Y/N

Enter the new node data
o
Do you want to continue?Y/N
Enter the new node data
r
Do you want to continue?Y/N
Enter the new node data
u
Do you want to continue?Y/N
Enter the new node data
y
Do you want to continue?Y/N
Enter the new node data
n
Do you want to continue?Y/N
Enter the new node data
q
Do you want to continue?Y/N
Enter the new node data
s
Do you want to continue?Y/N
Enter the new node data
v
Do you want to continue?Y/N

```

```

Address of d is 2052
Address of e is 2092
Address of f is 2012
Address of g is 2102
Address of h is 2062
Address of i is 2032
Address of j is 2112
Address of k is 2072
Address of l is 2122
Address of m is 2002
Address of n is 2202
Address of o is 2162
Address of p is 2142
Address of q is 2212
Address of r is 2172
Address of s is 2222
Address of t is 2132
Address of u is 2182
Address of v is 2232
Address of w is 2152
Address of x is 2242
Address of y is 2192
Address of z is 2252
Do you want to continue to go main menu?

```

Figure 17: Process flow

Rules sets	# of patterns	# of char	# of transitions	# of states	Memory (Bytes)	# of transitions	# of states	Memory (Bytes)	Memory Reduction	Our New Algorithm with Merged FSM
Oracle	337	11128	6793	6804	49267	4432	3846	30699	38%	490
Deleted	310	4636	3241	3251	22702	2541	2322	17776	22%	780
Exploit	160	2052	1561	1567	10545	1234	1135	8348	21%	560
Web-misc	156	1644	1420	1425	9592	1305	1247	8892	7%	740
Smtip	104	989	715	719	4653	625	600	4109	12%	1120
Misc	97	1403	1133	1137	7653	897	820	5846	24%	920
Ftp	96	466	402	406	2517	385	374	2442	30%	900
Other sets	957	14041	10912	10871	81768	9345	8777	70262	14%	1180
total rules	2217	36359	26177	26180	188697	20764	19121	148374	21%	6690
Reduction					21%					4%

Figure 18: Results Table of AC algorithm

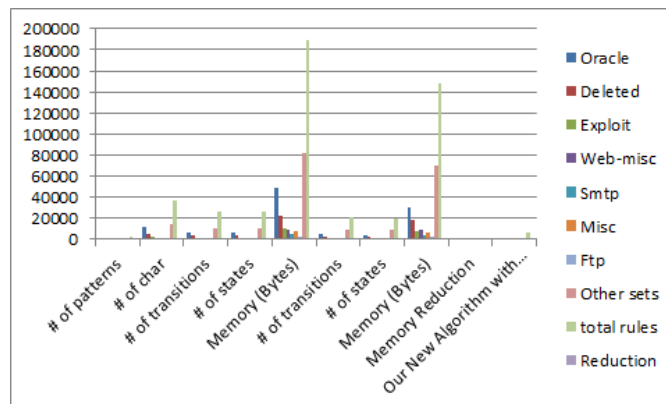


Figure 19: Results of the AC Algorithm

5 Conclusion and Future Enhancement

In this study, a pattern-matching technique that is more efficient in terms of memory usage is introduced. The proposed technique can considerably minimize the number of states and transitions by merging the states that are pseudo-equivalent during an attack. To add on, the presented work is complementary in nature to that of other approaches that focus on minimizing memory usage. The

WIDS is very useful for encrypting and decrypting the merged pattern-matching strings. It provides security against unauthorized attack. The future work focuses on constructing a B or B+ tree, which can further help in improving the memory usage by minimizing the memory address of the strings taken for input.

Acknowledgement: Not applicable.

Funding Statement: Not applicable.

Author Contributions: The authors confirm contribution to the paper as follows: Conceptualization, methodology and writing—original draft preparation: K. Padmavathi; supervision and writing—review and editing: Basheer Riskhan. All authors reviewed the results and approved the final version of the manuscript.

Conflicts of Interest: The authors declare no conflicts of interest to report regarding the present study.

References

1. Singh R, Kumar H, Singla RK, Ketti RR. Internet attacks and intrusion detection system: A review of the literature. *Online Inf Rev.* 2017;41(2):171-84.
2. Satam P, Hariri S. WIDS: An anomaly-based intrusion detection system for Wi-Fi (IEEE 802.11) protocol. *IEEE Trans Netw Serv Manag.* 2020;18(1):1077-91.
3. Ozkan-Okay M, Samet R, Aslan Ö, Gupta D. A comprehensive systematic literature review on intrusion detection systems. *IEEE Access.* 2021;9:157727-60.
4. Alenezi MN, Alabdulrazzaq H, Mohammad NQ. Symmetric encryption algorithms: Review and evaluation study. *Int J Commun Netw Inf Secur.* 2020;12(2):256-72.
5. Poh GS, Chin JJ, Yau WC, Choo KK, Mohamad MS. Searchable symmetric encryption: Designs and challenges. *ACM Comput Surv.* 2017;50(3):1-37.
6. Barengi A, Breveglieri L, Izzo N, Pelosi G. Exploring cortex-M microarchitectural side channel information leakage. *IEEE Access.* 2021;9:156507-27.
7. Nakahara H, Sasao T, Matsuura M. A regular expression matching circuit based on a decomposed automaton. In *International Symposium on Applied Reconfigurable Computing*, 2011 Mar 23, Berlin, Heidelberg, 2011, p. 16-28.
8. Hutchings BL, Franklin R, Carver D. Assisting network intrusion detection with reconfigurable hardware. In *Proceedings of 10th Annual IEEE Symposium on Field-Programmable Custom Computing Machines*, 2002 Apr 24, Napa, CA, USA, 2002, p. 111-120.
9. Moscola J, Lockwood J, Loui RP, Pachos M. Implementation of a content-scanning module for an internet firewall. In *11th Annual IEEE Symposium on Field-Programmable Custom Computing Machines*, 2003 Apr 9, Napa, CA, USA, 2003, p. 31-38.
10. Clark CR, Schimmel DE. Efficient reconfigurable logic circuits for matching complex network intrusion detection patterns. In *International conference on field programmable logic and applications*, 2003 Sep 1, Berlin, Heidelberg, 2003, p. 956-959.
11. Baker ZK, Prasanna VK. High-throughput linked-pattern matching for intrusion detection systems. In *Proceedings of the 2005 ACM symposium on Architecture for networking and communications systems*, 2005 Oct 26, New York, NY, USA, 2005, p. 193-202.
12. Lin CH, Huang CT, Jiang CP, Chang SC. Optimization of regular expression pattern matching circuits on FPGA. In *Proceedings of the Design Automation & Test in Europe Conference*, 2006 Mar 6, Munich, 2006, p. 1-6.
13. Pao D, Lin W, Liu B. A memory-efficient pipelined implementation of the aho-corasick string-matching algorithm. *ACM Trans Archit Code Optim.* 2010;7(2):1-27.
14. Aldwairi M, Conte T, Franzon P. Configurable string-matching hardware for speeding up intrusion detection. *ACM SIGARCH Comput Arch News.* 2005;33(1):99-107.
15. Tan L, Brotherton B, Sherwood T. Bit-split string-matching engines for intrusion detection and prevention. *ACM Trans Archit Code Optim.* 2006;3(1):3-4.

16. Jung HJ, Baker ZK, Prasanna VK. Performance of FPGA implementation of bit-split architecture for intrusion detection systems. In Proceedings 20th IEEE International Parallel & Distributed Processing Symposium, 2006 Apr 25, Rhodes, Greece, 2006, p. 8-14.
17. Piyachon P, Luo Y. Compact state machines for high performance pattern matching. In Proceedings of the 44th Annual Design Automation Conference, 2007 Jun 4, New York, NY, USA, 2007, p. 493-496.
18. Piyachon P, Luo Y. Design of high-performance pattern matching engine through compact deterministic finite automata. In Proceedings of the 45th Annual Design Automation Conference, 2008 Jun 8, New York, NY, USA, 2008, p. 852-857.
19. Cho YH, Mangione-Smith WH. A pattern matching coprocessor for network security. In Proceedings of the 42nd annual Design Automation Conference, 2005 Jun 13, New York, NY, USA, 2005, p. 234-239.
20. Cho YH, Mangione-Smith WH. Fast reconfiguring deep packet filter for 1+ gigabit network. In 13th Annual IEEE Symposium on Field-Programmable Custom Computing Machines (FCCM'05), 2005 Apr 18, Napa, CA, USA, 2005, p. 215-224.
21. Dharmapurikar S, Lockwood JW. Fast and scalable pattern matching for network intrusion detection systems. *IEEE J Sel Areas Commun.* 2006;24(10):1781-92.
22. Gokhale V, Kroep K, Rao VS, Verburg J, Yechangunja R. Tixt: An extensible testbed for tactile internet communication. *IEEE Internet Things J.* 2020;3(1):32-7.
23. Sourdis I, Pnevmatikatos D. Pre-decoded CAMs for efficient and high-speed NIDS pattern matching. In 12th Annual IEEE symposium on field-programmable custom computing machines, 2004 Apr 20, Napa, CA, USA, 2004, p. 258-267.
24. Yu F, Katz RH, Lakshman TV. Gigabit rate packet pattern-matching using TCAM. In Proceedings of the 12th IEEE International Conference on Network Protocols, 2004 Oct 8, Berlin, Germany, 2004, p. 174-183.
25. Dharmapurikar S, Krishnamurthy P, Sproull T, Lockwood J. Deep packet inspection using parallel bloom filters. In 11th Symposium on High Performance Interconnects, 2003 Aug 20, Stanford, CA, USA, 2003, p. 44-51.
26. Lockwood JW, Moscola J, Kulig M, Reddick D, Brooks T. Internet worm and virus protection in dynamically reconfigurable hardware. In Proceedings of the Military and Aerospace Programmable Logic Device Conference, 2003 p. 1-8.